

Special Topic

Game Engine & Math/Physics

Horazon

Game Development Workshop

About Me

Shih-Ming Chang (Horazon)

- **Current Position:** Assistant Professor, Dept. of Multimedia Game Development and Application, Hungkuang University
- **Expertise:** Game Development, Programming

Dept. of Multimedia Game Development and Application

We are dedicated to cultivating professional talents with "Digital Creativity" and "Technical Strength."

- **Core Areas:** Games, Animation, Comics, Streaming, E-sports
- **Special Facilities:** E-sports Arena, Motion Capture Room, Game Development Lab
- **Goal:** Connect students with international industry trends through practice and collaboration.

Here, it's not just about playing games, it's about **Creating Games!**

Topics

- Understand what a game engine is and why we need it.
- Recognize the differences between common game engines on the market.
- Learn about the "Four Core" functions of a game engine.
- Connection between game engines and high school mathematics.
- Connection between game engines and high school physics.

What is a Game Engine?

A game engine is the core software for developing games; think of it as a **specialized toolbox**.

- Provides ready-made and verified code and tool modules.
- Integrates various systems like graphics, sound, and physics.

It allows us to focus on designing gameplay without having to write low-level technology.

How Exhausting is it to Develop from Scratch?

Without using an engine, all programs need to be reinvented.

- **Massive Workload:** Writing code for "stopping when hitting a wall" could take months.
- **High Technical Bar:** Requires deep knowledge of calculus and computer graphics systems.

Developing completely from scratch is something only a few top engineers can achieve.

Core Benefits of Using an Engine

- **Save Time:** Provides ready-made general functions that take effect with just a click.
- **Lower Barriers:** Has a simple visual mouse interface that non-technical personnel can operate.
- **Cross-platform Output:** Develop once, and easily publish to mobile phones or game consoles.

Hardly anyone writes code from zero now; using an engine is the most basic "common sense" in the industry.

Three Common Game Engines

There are many different engine tools to choose from on the market.

- Their biggest differences lie in their main features and learning difficulty.
- Currently, the three most well-known engines are: **Unity, Unreal, Godot.**

Choosing the right engine from the start according to your project scale and needs is very important.



The most widely used and resource-rich software in the industry.

- **Features:** Developed using **C#**, has massive learning resources, and is versatile for both 2D and 3D.
- **Target:** Very suitable for beginners and the first choice for small independent teams.
- **Known Games:** *Genshin Impact*, *Pokemon GO*, *Hollow Knight*.

Unreal Engine



World-renowned for its powerful "top-tier visual graphic performance."

- **Features:** Developed using C++ or the code-free "Blueprints," with built-in god-tier lighting/shadows.
- **Target:** Large projects and 3A game companies pursuing extreme graphics.
- **Known Games:** *Fortnite*, *Black Myth: Wukong*.

Godot Engine



An emerging "open-source engine" beloved by independent developers.

- **Features:** Completely free with zero royalty, extremely small software size, and very fast execution.
- **Target:** Solo developers or teams making lightweight casual games with simple mechanics.
- **Known Games:** *Brotato*, *Dome Keeper*.

Four Core Functions of an Engine

The core functions of an engine include at least the following:

- 1. Rendering
- 2. Physics
- 3. Audio
- 4. Scripting

Core 1: Rendering

The rendering system is responsible for processing and **drawing world data in real-time** for our eyes to see.

- Handles stacking and rotation of 2D images.
- Handles textures of 3D models and shadows after lighting is applied.
- Key Task: Forced maintenance of rapid frame output (e.g., 60 FPS smoothness).

The better the rendering technology, the more gorgeous and realistic the game graphics look.

Core 2: Physics Calculation

The goal of the physics system is to make virtual objects on the screen react like they are in the real world.

- **Weight and Gravity:** Controls how fast everything falls to the ground.
- **Collision Detection:** Decides how two cars should rotate and bounce off when crashing at high speed.
- Usually uses third-party physics engines like Box2D or PhysX.

The engine calculates these automatically, so developers don't need to write difficult physics formulas all day long.

Core 3: Audio Management

The audio system helps us manage all background environment music and sound effects systematically.

- Handles temporary sounds like sudden gunfire or explosions.
- Automatically adjusts volume based on the distance between the player and the sound source.
- Directly helps create a sense of 3D spatial sound when playing with headphones.

Core 4: Scripting System

The code files we write using letters and digits are the **scripts** that control everything.

- Monitors whether a player has pressed a button and then controls the character's jumping behavior.
- Calculates the remaining health of monsters and the player's total score.
- Responsible for telling the engine what sound or animation to play when a monster dies.

Code is the central nervous system of the game's body.

Other Powerful Engine Tools

Besides the four cores, engines provide many modules to make games "come alive":

- **Animation System:** Manages character rigs and action transitions.
- **Camera System:** Acts as a virtual cinematographer, responsible for view tracking, camera shake, and post-processing effects.
- **UI System:** Create health bars, mini-maps, and menus.
- **VFX/Particles:** Simulate fire, smoke, explosions, or glowing magic.
- **AI Navigation:** Give NPCs "intelligence" so they can automatically calculate shortest paths and avoid obstacles.

These integrated tools allow developers to build grand worlds quickly by "stacking blocks."

Game Development & High School Math

Even with engines reducing the burden, game logic still relies on pure foundational high school math.

- **Helping the Computer Control Space:** Computers only understand numbers; math is responsible for specifying "where is the direction and target."
- **No Need to Solve Manually:** You don't need pen and paper; often you just need to call the engine's built-in math commands.

The focus now is: "In what situation, do you know which math mechanism to choose."

Coordinate System

The only role of coordinates is to clearly locate the **absolute precise position** of every single thing in the world using numbers.

- **2D Coordinates (X, Y):** In a flat game space, X-axis handles horizontal width, and Y-axis handles vertical height.
- **3D Coordinates (X, Y, Z):** Once depth is added (Z-axis represents front-back distance), the game becomes three-dimensional.
- **No Coordinates, No Existence:** Even the center of the game map is marked as the origin position `(0, 0, 0)`.

Vector

An arrow that possesses both "**Magnitude (Size)**" and a "**Direction**."

- **Movement in Games:** Characters walk because they are pushed by a velocity vector.
- **Direction:** Wherever the arrow points, that's where the character is pushed.
- **Magnitude:** The longer the arrow (the larger the number), the longer and faster the step taken.

Whenever you see dynamic position changes on the screen, vectors are being manipulated behind the scenes.

Simple Vector Addition and Subtraction

Even if your math isn't great, most operations can be solved with basic vector addition and subtraction.

- **Addition for Moving Forward:** Add "this step's velocity vector" to the "current position" = update to the new position.
- **Subtraction for Turning:** Enemy's position - Your position = get the direction pointing from you to the enemy.
- **Calculate Distance:** Ask the engine for the length of the vector segment, which results in the actual linear distance between two points.

Trigonometry: Angles and Circular Motion

Whenever you encounter features related to **regularly rotating changes** or **angle limit detection**, you'll use trigonometry.

- **Calculate Dead Zones:** Calculate the angle of the guard's flashlight cone to see if the player is lucky enough to hide in a dead zone.
- **Oscillating Dynamic:** Use the Sine wave that naturally moves from 1 back to -1 for repeated wave rhythms like a floating ship.
- **Planetary Orbits:** Overlay functions to accurately generate tracking orbital points that rotate perfectly around a central star.

Matrix Basics

This is a tool that stuffs a whole bunch of numbers into a table to be changed together. It is also the arithmetic core of everything drawn on the screen.

- **Hero Behind Scaling and Deformation:** For a giant monster's whole body to tilt and turn, it's the matrix calculation at the bottom.
- **Screen Projection:** Compress data from beautiful 3D space environments to display as flat pixels on a computer screen.
- **GPUs Specialization:** The graphics card you bought is hardware whose primary "day job" is to kill tens of thousands of matrix operations in a single second.

Transformation Matrix: TRS (Translation, Rotation, Scale)

In 3D space, every object has a "Transformation" matrix that determines how it appears.

- **Translation:** The coordinate position (x, y, z) in space.
- **Rotation:** The direction the object faces. Engines often use "Quaternions" to calculate this to avoid gimbal lock.
- **Scale:** The volume/size multiplier of the object.

Although games are linked to matrices, we don't need to write matrices ourselves; the engine handles them automatically.

Translation and Scaling Matrices

Translation Matrix

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Scaling Matrix

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation Matrix

Rotation around Y-axis Matrix

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Quaternion (w,x,y,z) to Matrix

$$M_q = \begin{bmatrix} 1 - 2y^2 - 2z^2 & 2xy - 2zw & 2xz + 2yw & 0 \\ 2xy + 2zw & 1 - 2x^2 - 2z^2 & 2yz - 2xw & 0 \\ 2xz - 2yw & 2yz + 2xw & 1 - 2x^2 - 2y^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Why Do We Need a Physics Engine?

A physics engine is like bringing a "High School Physics Lab" into the computer, adding a **sense of interactive reality** to games.

- **Automation:** No need to write formulas manually; the engine handles gravity, friction, and collision mechanics.
- **Connect with High School Physics:** This is a digital sandbox where you can experiment with Newton's laws and energy conservation.
- **Enhanced Fun:** Let players push wooden boxes or blast enemies away just like in reality, increasing the tactile feedback.

Digital Physics Experiments: Connecting with High School Courses

You can treat a game engine as the most powerful experimental equipment.

- **Verify Formulas:** Directly simulate free fall or pendulum motion in the engine to see if the values match the formulas.
- **Risk-Free Experiments:** Freely adjust gravity constants (e.g., simulate lunar gravity) and observe the changes in object motion.
- **Visualization:** Transform dry physics problems into vivid character actions or explosive special effects.

Rigidbody and Collision: Foundations of Mechanics

Simulation

To move an object in the computer, you first need to add two key components.

- **Rigidbody:** Gives the object "Mass." With it, the object will be affected by gravity and generate inertia.
- **Collider:** Defines the "Boundary" of the object. Ensures two balls bounce off when they hit instead of passing through each other.
- **Friction:** Adjust surface friction. For example, sliding on "grass" stops faster than on "ice."

Projectile Motion: Force and Flight Trajectories

To simulate an arrow or cannonball path, you only need to set the initial force in the engine.

- **Give Impulse:** Apply an instantaneous thrust to a Rigidbody as the initial velocity.
- **Gravity Effect:** After the object takes off, the engine continuously applies downward gravitational acceleration g .
- **Automatic Parabola:** Combining initial velocity and gravity perfectly presents a projectile motion trajectory.

Restitution: Simulating Different Material Reactions

Adjusting "Bounciness" can simulate the degree of kinetic energy recovery after a collision.

- **Restitution (Bounciness):** Determines the bounce height after collision.
- **Material Differences:**
 - **High Bounciness:** Trampolines, rubber balls (recovers most kinetic energy).
 - **Low Bounciness:** Sandbags, floor (energy is almost absorbed).
- **Implementation:** Easily adjust through `Physics Material1` in the engine.

Unity: The Most Powerful C# Development Platform

Unity is a development environment centered around "Components," using the C# language.

- **Component-based Architecture:** No need for super long code; functionality is broken into small parts attached to objects.
- **King of Cross-Platform:** Write code once, publish to iOS, Android, PC, Switch, PS5.
- **Digital Twins, VR/AR:** Besides games, Unity is the first choice for Metaverse and industrial simulations.

```
float speed = 10f;
void Update(Time.deltaTime)
{
    // Move forward every frame based on time delta
    transform.Translate(Vector3.forward * speed * Time.deltaTime);
}
```

AI-Era Coding Revolution

In modern development environments, code "syntax" is no longer the main barrier.

- **AI-Assisted Development:** Use tools like GitHub Copilot or Cursor; AI predicts intent and generates boilerplate.
- **From "Typing" to "Conversing":** The key is understanding logic and how to give Prompts, letting AI handle tedious coding.
- **Rapid Prototyping:** With AI assistance, a feature that used to take three days can now be completed in ten minutes.

Teacher's Advice: Learning programming is no longer about rote memorization of syntax, but learning how to collaborate with AI to solve problems.

GDevelop 5: Top Choice for No-Code Engines



If you absolutely don't want to touch programming syntax, [GDevelop 5](#) is the best visual development tool.

- **100% Visual:** Complete logic through mouse clicks without writing a single line of code.
- **Cross-platform & Web Support:** Develop directly in the browser or output as standalone software for mobile or PC.
- **Powerful Extensions:** Built-in "Behaviors" (Behaviors) that can be dragged and dropped to finish platformer movement.

Event-Driven Logic (Event System)

The soul of GDevelop lies in its "Event Editor," which is also the training ground for development logic.

- **Condition:** When what happens? (e.g., Player presses the Space key)
- **Action:** Then execute what action? (e.g., Character plays jump animation and surges upward)
- **Logic Manifestation:** Although you don't write code, it still requires strong logical judgment and process scheduling.

For game designers and planners, this is the sharpest tool for quickly realizing ideas and verifying gameplay.

Thank you

Introducing students to game engines early may spark interest in mathematics or physics.

Moving from this classroom to the computer lab.

- Turn on the computer

- Prepare a Google account

- Try operating Unity or GDevelop 5.